Generation rules for proof obligations in the chapter "The Event-B Modelling Language"

Analysis and precise description

Dominique Méry

November 6, 2026

Abstract

This document gives a precise description of the rules for generating the verification conditions (*proof obligations*, written POs below) presented in the Event-B chapter [1] of the book *Modelling Software-based Systems: Correct by Construction Paradigm*[2]. For each rule, it gives its origin in the text, the model element that triggers it, the exact form of the sequent it produces, its naming convention (Rodin style), its justification and, when the chapter discusses them, the limitations and adjustments of Rodin.

Contents

1 Purpose, scope and method **2**

2 Framework, notation and basic rules **2**
2.1 Event-based model . . . 2
2.2 Normalised form of an event . . . 2
2.3 Notation used in the sequents . . . 3
2.4 Simplification of typing . . . 3
2.5 The two sequent decomposition rules . . . 3
2.6 Naming convention . . . 4

3 Foundations: where the POs come from **4**
3.1 Induction principle . . . 4
3.2 Fixed-point characterisation . . . 4
3.3 Termination and feasibility of an event . . . 4
3.4 Equivalence of the *wp* and relational styles . . . 4

4 Rules for a context **5**

5 Rules for an abstract machine **6**
5.1 Event status and variants . . . 8

6 Rules for refinement **9**
6.1 Reference definition and derivation . . . 9
6.2 Rules generated for a concrete event . . . 10

7 Deadlock freedom (semantics) **13**

8 Summary table **13**

1 Purpose, scope and method

The chapter under analysis (compiled version `main-C1.pdf`) presents the POs in three stages:

1. **semantic foundations** (§1.2): transition system, induction principle, fixed-point characterisation, equivalence of the *wp* and relational styles;
2. the **POs of a context and of an abstract machine** (§1.3);
3. the **POs of refinement** (§1.4), complemented by event merging and by a semantic discussion of deadlock freedom.

Section 1.5 (abacus, "Rule 1" and "Rule 2") describes the rules for using an abacus and defines no new PO generation rule. It illustrates how the rules described here are applied, and is therefore outside the scope of this document.

The text always follows the same generation method:

(M1) Global condition. Start from a global semantic condition (preservation of an invariant, refinement in *wp*, etc.).

(M2) Sequent form. Rewrite it as a sequent by removing the universal quantifications over the state, which become free variables.

(M3) Decomposition. Apply two sequent-calculus rules (§2.5) to obtain elementary sequents.

(M4) Elimination of existentials. When possible, replace an existential quantification in the conclusion by a *witness* supplied by the modeller, which gives rise to a PO for the feasibility of the witness.

(M5) Naming. Each elementary sequent is named after the label $\ell(text)$ of the formal text (axiom, invariant, guard, action, theorem) that produced it, in the form element/label/TYPE.

2 Framework, notation and basic rules

2.1 Event-based model

An event-based model of a system S (def. *event-based model*) is a quintuple

$$EM(S) = (AX(s,c),\ \{x\},\ \mathcal{V}alues,\ Init(x),\ \{e_0,\ldots,e_n\})$$

where s are the basic sets, c the constants, $AX(s,c)$ the axiomatic theory (the context), x the flexible variables, *Init* the initial predicate and e_i the events. The event e_0 is skip. The chapter now writes states σ, σ', the letter s being reserved for the basic sets (l. 407), and writes the refinement events in sans-serif type, a and c, to distinguish the concrete event c from the list c of constants (l. 1972).

2.2 Normalised form of an event

An event is written

$$\mathsf{e} \mathrel{\widehat{=}} \textbf{any } u \textbf{ where } G(u,s,c,x) \textbf{ then } x :| BAP(u,s,c,x,x') \textbf{ end}$$

and its before-after relation is

$$BA(\mathsf{e})(s,c,x,x') \mathrel{\widehat{=}} \exists u \cdot G(u,s,c,x) \wedge BAP(u,s,c,x,x').$$

The global transition relation is $Next \mathrel{\widehat{=}} \bigvee_i BA(e_i)$, and $Next^\star$ is its reflexive transitive closure, defined as the *least* relation satisfying the recursive equation of the chapter (l. 492).

2.3 Notation used in the sequents

$AX(s,c)$	conjunction of the axioms of the context that is seen (sees) and of the contexts it extends
$Th(s,c)$	theorems of the context that precede the theorem under consideration
$Th_i(s,c)$	theorems located *before* the machine theorem th_i (for $i=1$, only those of the context)
$I(s,c,x)$	invariant $I_1 \wedge \cdots \wedge I_r$ of the machine, labelled $inv_1, \ldots, inv_r$
$SAFE_i(s,c,x)$	safety properties (clause THEOREMS), $i \in 1..h$; $A = \bigwedge_i SAFE_i$
$initBAP$	before-after predicate of the initialisation, split by actions act_i; $Init(s,c,x) \equiv initBAP(s,c,x)$
$G,\ BAP$	guard and before-after predicate of the abstract event
$J(s,c,x,y)$	gluing invariant $J_1 \wedge \cdots \wedge J_r$, labelled jnv_i, linking x (abstract) and y (concrete)
$H,\ CBAP$	guard and before-after predicate of the concrete event (parameters v)
$WP(u,s,c,v,y)$	witness for the abstract parameter u (*Witness Parameter*)
$WV(v,s,c,y',x')$	witness for the abstract after-value x' (*Witness Variable*); at initialisation, $WV(s,c,y',x')$
$varexp$	the single variant of the machine (natural number or finite set)

2.4 Simplification of typing

The semantic conditions are first written in the form $AX(s,c) \vdash \forall x \in \mathcal{V}alues \cdot P(x)$. Since x has no free occurrence in $AX(s,c)$, they are rewritten as $AX(s,c), x \in \mathcal{V}alues \vdash P(x)$. The typing hypothesis $x \in \mathcal{V}alues$ is then assumed to be carried by the invariant and the initialisation, and is omitted.

2.5 The two sequent decomposition rules

The whole generation rests on two rules (property "simplification of sequents" and property "Introduction/Elimination of connector $\wedge$"):

Rule S1: implication introduction

$$AX(s,c), \mathcal{H} \vdash P_1 \Rightarrow P_2 \Rightarrow \ldots \Rightarrow P_n \Rightarrow Q \quad \text{is equivalent to} \quad AX(s,c), \mathcal{H}, P_1, \ldots, P_n \vdash Q.$$

Rule S2: splitting a conjunction

$$\Gamma \vdash \bigwedge_{i \in 1..t} P_i \quad \text{is equivalent to} \quad \Gamma \vdash P_i \ \text{ for all } i \in 1..t.$$

This is the rule that produces *one PO per label*: one per invariant, per theorem, per action, etc.

Two implicit rules complete these explicit ones. The text uses them in the derivations without naming them:

- **S3, existential in a hypothesis**: $\Gamma, \exists v \cdot P(v) \vdash Q$ is equivalent to $\Gamma, P(v) \vdash Q$ if v is not free in Γ, Q (the variable becomes a free parameter of the sequent);
- **S4, existential in the conclusion by a witness**: if $\Gamma, W(u) \vdash P(u)$ and $\Gamma \vdash \exists u \cdot W(u)$, then $\Gamma \vdash \exists u \cdot P(u)$. This is the basis of the WFIS POs (§6.2); the second premise is essential.

2.6 Naming convention

POs are named as in Rodin: object/label/TYPE, where *object* is an event (or INITIALISATION), a theorem or an axiom, *label* is the label of the invariant, action, guard or witness concerned, and *TYPE* is one of the following suffixes: WD, THM, INV, FIS, GRD, SIM, WFIS, WWD, NAT, FIN, VAR, MRG, DLF. For variants, Rodin inserts no label: e/NAT, e/FIN, e/VAR. The suffixes -WIT, -P and -V (GRD-WIT, WFIS-P, ...) belong to the chapter: Rodin produces GRD and SIM with the witness predicates as hypotheses (l. 2287).

3 Foundations: where the POs come from

3.1 Induction principle

Induction principle (property *induction principle*)

$P(x)$ is a safety property of S if, and only if, there exists $I(x)$ such that, for all x, x':

$$(1)\ \mathit{Init}(x) \Rightarrow I(x) \qquad (2)\ I(x) \Rightarrow P(x) \qquad (3)\ I(x) \wedge \mathit{Next}(s,c,x,x') \Rightarrow I(x').$$

The *equivalence of induction principles* replaces (3) by one condition per event: $\forall i \cdot I(x) \wedge BA(e_i)(s,c,x,x') \Rightarrow I(x')$. This is rule S2 applied to the disjunction in the hypothesis, since $(\bigvee_i A_i) \Rightarrow B \equiv \bigwedge_i (A_i \Rightarrow B)$. This yields the three families of POs of the definition "Verification conditions for a system S" (where the safety property is written $A(x)$):

$$AX(s,c) \vdash \mathit{Init} \Rightarrow I, \qquad AX(s,c) \vdash I \wedge BA(\mathsf{e}) \Rightarrow I'\ (\forall \mathsf{e}), \qquad AX(s,c) \vdash I \Rightarrow A.$$

3.2 Fixed-point characterisation

Let $F = \lambda X \cdot \mathit{INIT} \cup \mathit{Next}[X]$ on the lattice $(\mathcal{P}(\mathcal{V}alues), \subseteq)$. The *inductive invariants* are exactly the sets I such that $F(I) \subseteq I$ (pre-fixed points), that is, $\mathit{INIT} \subseteq I$ and $\mathit{Next}[I] \subseteq I$; they are not only the solutions of $X = F(X)$ (l. 707). The least fixed point $\mathit{lfp}(F)$, the set of reachable states, is their intersection (Knaster–Tarski): every inductive invariant *contains* the reachable states. The POs therefore only check that a *proposed* invariant is indeed inductive: this is the correct-by-construction approach.

3.3 Termination and feasibility of an event

For an event, $\mathsf{trm}(\mathsf{e}) \mathrel{\widehat{=}} [\mathsf{e}]\mathit{true}$ is always *true* (under the WD conditions): an event always terminates. Only its feasibility is at stake. The chapter distinguishes two predicates:

$$\mathsf{fis}(\mathsf{e})(s,c,x) \equiv \exists u \cdot \big(G(u,s,c,x) \wedge \exists x' \cdot \mathit{BAP}(u,s,c,x,x')\big) \quad \text{(global feasibility: the actual guard)},$$
$$\mathsf{Fis}(\mathsf{e})(s,c,x) \mathrel{\widehat{=}} \forall u \cdot \big(G(u,s,c,x) \Rightarrow \exists x' \cdot \mathit{BAP}(u,s,c,x,x')\big) \quad \text{(per-parameter feasibility)}.$$

The obligation FIS requires $I \Rightarrow \mathsf{Fis}(\mathsf{e})$: it rules out the *hidden miracle*, a guard that holds while the action produces no state. The name FIS is reserved for the proof obligation.

3.4 Equivalence of the *wp* and relational styles

For an event e and a property P:

$$[\mathsf{e}]P(s,c,x) \equiv \forall u \cdot \big(G(u,s,c,x) \Rightarrow \forall x' \cdot (\mathit{BAP}(u,s,c,x,x') \Rightarrow P(s,c,x'))\big).$$

The *wp* condition (Atelier B style) $\vdash I \Rightarrow \mathsf{Fis}(\mathsf{e}) \wedge [\mathsf{e}]I$ is equivalent to the pair of relational conditions (Rodin style), free variables being universally quantified (property "*wp* and relational styles" of the chapter, with its proof):

$$(\text{FIS})\ \ I \wedge G \Rightarrow \exists x' \cdot \mathit{BAP}, \qquad (\text{INV})\ \ I \wedge G \wedge \mathit{BAP} \Rightarrow I(x').$$

Consequence: invariant preservation always splits into an *induction step* (INV) and a *feasibility proof* (FIS). The transformer [e] alone says nothing about feasibility: a parameter value with no next state satisfies every postcondition. This is why Fis(e) appears in the *conclusion* of the *wp* condition, not as a hypothesis. The chapter's definition "verification condition e" (def. `def:poe`) is $AX(s,c) \vdash I \Rightarrow \mathsf{Fis}(\mathsf{e}) \wedge [\mathsf{e}]I$.

4 Rules for a context

R1 axm/WD, th/WD, expr/WD: well-definedness of an expression

Trigger. Any expression or predicate *expr* written in the model (axiom, theorem, invariant, guard, action, variant, witness).

PO produced.

$$AX(s,c) \vdash WD(expr)$$

(in a machine, the hypotheses also include the invariant and, where relevant, the guards; see R8).

Computing *WD*. *WD* is defined by induction on the syntax, using three tables (predicates, unary/binary expressions, other expressions). The significant cases are:

e	$WD(e)$
$P \wedge Q,\ P \Rightarrow Q$	$WD(P) \wedge (P \Rightarrow WD(Q))$
$P \vee Q$	$WD(P) \wedge (P \vee WD(Q))$
$P \Leftrightarrow Q$	$WD(P) \wedge WD(Q)$
$\neg P$	$WD(P)$
$\forall L \cdot P,\ \exists L \cdot P$	$\forall L \cdot WD(P)$
$\lambda P \cdot Q \mid E$	$\forall \mathcal{F}_P \cdot WD(Q) \wedge (Q \Rightarrow WD(E))$
$\bigcap L \cdot P \mid E$	$\forall L \cdot (WD(P) \wedge (P \Rightarrow WD(E))) \wedge \exists L \cdot P$
$\bigcap E \mid P$	$\forall \mathcal{F}_E \cdot (WD(P) \wedge (P \Rightarrow WD(E))) \wedge \exists \mathcal{F}_E \cdot P$
$F(E)$	$WD(E) \wedge WD(F) \wedge E \in \mathrm{dom}(F) \wedge F \in S \mathrel{+\!\!\!\rightarrow} T$
E/F	$WD(E) \wedge WD(F) \wedge F \neq 0$
$E \bmod F$	$WD(E) \wedge WD(F) \wedge 0 \leq E \wedge 0 < F$
E^F	$WD(E) \wedge 0 \leq E \wedge WD(F) \wedge 0 \leq F$
$\mathrm{card}(E)$	$WD(E) \wedge \mathrm{finite}(E)$
$\mathrm{inter}(E)$	$WD(E) \wedge E \neq \varnothing$
$\min(E)$	$WD(E) \wedge E \neq \varnothing \wedge \exists v \cdot \forall u \cdot u \in E \Rightarrow v \leq u$
$\max(E)$	$WD(E) \wedge E \neq \varnothing \wedge \exists v \cdot \forall u \cdot u \in E \Rightarrow v \geq u$
E op F (relations, $+$, $\cup$, ...)	$WD(E) \wedge WD(F)$
identifiers, literals, $\mathbb{N}, \mathbb{Z}, \varnothing$	$\top$

In the λ row, P is the pattern (the bound variables) and Q the predicate; $\mathcal{F}_X$ denotes the variables bound by X.

Principle. Connectives are evaluated *from left to right and lazily*: in $P \wedge Q$, the well-definedness of Q may assume P. This is why the order in which axioms, invariants and guards are written affects the POs produced (see R8).

Example from the text. In this example, s is the sum function of the context, not the list of sets: $WD(\forall k \cdot k \in \mathbb{N} \Rightarrow 2 * s(k) = k * k + k) = \forall k \cdot k \in \mathbb{N} \Rightarrow k \in \mathrm{dom}(s) \wedge s \in \mathbb{Z} \mathrel{+\!\!\!\rightarrow} \mathbb{Z}$.

R2 th/THM: context theorem

Trigger. A theorem labelled th in a context (or a property on s, c placed among the theorems of a machine).

PO produced (l. 1293).

$$AX(s,c),\ \ Th(s,c) \vdash th$$

where $Th(s,c)$ denotes the theorems of the context that precede th.

Justification. The THEOREMS clause contains consequences of the axiomatic theory: they must be *proved*, not *assumed*. The consistency of the theory itself is not checked by any PO; the chapter recommends checking it, for instance with ProB.

5 Rules for an abstract machine

R3 th_i/THM: safety property (machine theorem)

Trigger. A theorem $th_i : SAFE_i(s,c,x)$ of the machine, $i \in 1..h$.

Derivation. $AX(s,c) \vdash I \Rightarrow \bigwedge_i SAFE_i \xrightarrow{\text{S1}} AX(s,c), I \vdash \bigwedge_i SAFE_i \xrightarrow{\text{S2}}$ one PO per i, to which the theorems already established are added.

PO produced (for each i).

$$AX(s,c),\ \ Th_i(s,c),\ \ I(s,c,x) \vdash SAFE_i(s,c,x)$$

Role. It separates the *inductive* properties (invariants, which require an induction step) from the safety properties that *follow* from them (theorems). In the SAFETY example, $x \leq 0$ is not inductive, but it follows from the inductive invariant $x = -1$.

R4 INITIALISATION/inv_i/INV: establishing the invariant

Trigger. The INITIALISATION event, written $x :| \ initBAP(s,c,x')$, and each invariant inv_i.

Derivation. $AX(s,c) \vdash Init \Rightarrow I \xrightarrow{\text{S1}} AX(s,c), Init \vdash I \xrightarrow{\text{S2}}$ one PO per inv_i, stated by Rodin with the after-values x'.

PO produced (l. 1577).

$$AX(s,c),\ \ initBAP(s,c,x') \vdash I_i(s,c,x')$$

Remark. The initialisation has no before-state: neither the invariant nor a typing hypothesis on x appears as a hypothesis.

R5 INITIALISATION/act_i/FIS: feasibility of the initialisation

Trigger. Each *non-deterministic* action $act_i : x_i :| \ initBAP_i(s,c,x_i')$ of the initialisation, the x_i forming a *partition* of x.

Derivation. $AX(s,c) \vdash \exists x' \cdot initBAP(s,c,x')$ with $initBAP \equiv \bigwedge_i initBAP_i$. Since the x_i are disjoint, the existential distributes: $\exists x' \cdot \bigwedge_i initBAP_i(x_i') \equiv \bigwedge_i \exists x_i' \cdot initBAP_i(x_i')$, and S2 is then applied.

PO produced.

$$AX(s,c) \vdash \exists x_i' \cdot initBAP_i(s,c,x_i')$$

Role. It guarantees that the model has at least one initial state. When the action gives no explicit value (for instance $acc :| acc' \in P \to B \land acc' \subseteq a$), a witness constant $acc0$ is introduced in the context. The chapter points out that this only moves the difficulty: the axiom on $acc0$ is consistent only if a contains a total function from P to B (l. 1617). For a deterministic assignment $x := e$, no FIS PO is produced; only the WD PO for e remains.

R6 e/inv$_i$/INV: invariant preservation by an event

Trigger. Each event e (other than the initialisation) and each invariant inv_i.

Derivation. From (INV): $AX(s,c) \vdash I \land BA(\mathsf{e}) \Rightarrow I'$, unfold BA, eliminate $\exists u$ in the hypothesis (S3), apply S1, then S2.

PO produced.

$$AX(s,c),\ I(s,c,x),\ G(u,s,c,x),\ BAP(u,s,c,x,x') \vdash I_i(s,c,x')$$

Remark. The *whole* invariant is a hypothesis; only the conclusion is split. In the SAFETY example, the PO for $x = -1$ holds because the hypotheses are inconsistent ($x = -1$, $x \geq 0$). With $x \leq 0$, one would obtain the false PO $\cdots \vdash 1 \leq 0$, which shows that $x \leq 0$ is not inductive.

R7 e/act$_i$/FIS: feasibility of an event

Trigger. Each *non-deterministic* action $act_i : x_i :| BAP_i(u,s,c,x,x_i')$ (or $x_i :\in E$) of the event e.

PO produced (l. 1707).

$$AX(s,c),\ I(s,c,x),\ G(u,s,c,x) \vdash \exists x_i' \cdot BAP_i(u,s,c,x,x_i')$$

Role. It corresponds to the Fis(e) part of the *wp* condition: when the guard holds, the event can actually produce a next state. As for the initialisation, the existential distributes over the actions, which act on disjoint variables, hence one PO per action. For $x := e$, the existence of $x' = e$ is trivial and no FIS PO is produced (l. 1625).

In Rodin: limitations and adjustments

Box *feasibility in Rodin* of the chapter (l. 1630–1643):

- FIS is generated only for non-deterministic actions, $x :\in E$ and $x :| P(x,x')$; for $x :\in E$ it reduces to $E \neq \varnothing$; an action $x := e$ produces only WD;
- feasibility is split action by action; this is sound because the actions modify disjoint lists of variables;
- limitation: the feasibility of an action is proved in isolation, and an unprovable FIS PO does not say whether to strengthen the guard or weaken the action.

R8 e/grd$_i$/WD, e/act$_i$/WD and INITIALISATION/act$_i$/WD: well-definedness of guards and actions

Trigger. Each guard and each action of an event, and each action of the initialisation.

POs produced (l. 1723).

$$AX(s,c),\ I(s,c,x),\ G_1,\ldots,G_{i-1} \vdash WD(G_i(u,s,c,x)) \qquad (\mathsf{e/grd}_i/\mathsf{WD})$$
$$AX(s,c),\ I(s,c,x),\ G(u,s,c,x) \vdash WD(BAP_i(u,s,c,x,x_i')) \qquad (\mathsf{e/act}_i/\mathsf{WD})$$
$$AX(s,c) \vdash WD(initBAP_i(s,c,x_i')) \qquad (\mathsf{INITIALISATION/act}_i/\mathsf{WD})$$

Role. They rule out "silly" expressions (in Lamport's sense), such as a division by zero. The guards that precede G_i are hypotheses: the list of guards is read as the conjunction $G_1 \wedge \cdots \wedge G_k$, to which the rule $WD(P \wedge Q) = WD(P) \wedge (P \Rightarrow WD(Q))$ is applied. This allows one to write the two guards $grd_1 : x \in \mathrm{dom}(f)$ and $grd_2 : f(x) > 0$.

In Rodin: limitations and adjustments

Box *well-definedness in Rodin* of the chapter (l. 1735–1748):

- the same left-to-right reading applies to every labelled list: an axiom, an invariant or a guard is proved well defined with those that precede it, and an action with all the guards;
- order matters: exchanging grd_1 and grd_2 above yields $AX(s,c), I \vdash x \in \mathrm{dom}(f)$, which is in general not provable;
- within a single predicate, $\wedge$ is not commutative for WD: $x \in \mathrm{dom}(f) \wedge f(x) > 0$ is well defined, $f(x) > 0 \wedge x \in \mathrm{dom}(f)$ is not;
- Rodin discharges most WD POs automatically: an undischarged WD PO usually points to a missing or misplaced hypothesis.

5.1 Event status and variants

Each event has a status: *ordinary*, *convergent* or *anticipated*. The text justifies the variant POs by the following liveness rule. If $R_{n+1} \rightsquigarrow R_n$ for all n, if $P \wedge I \Rightarrow \exists n \cdot R_n$ and if $R_0 \Rightarrow Q$, then $P \rightsquigarrow Q$, with $R_n \widehat{=} I \wedge varexp \leq n$ (l. 1799) (the variant may decrease by more than one unit). The POs ensure that a convergent event strictly decreases a well-founded quantity, hence that convergent events cannot occur forever. To conclude $P \rightsquigarrow Q$, a convergent event must also remain enabled as long as Q does not hold: this follows from deadlock freedom or from a fairness assumption.

R9 e/NAT and e/FIN: the variant is well founded

Trigger. A *convergent* (or anticipated) event e. A machine has a single variant (clause variant).

PO produced (numeric variant).

$$AX(s,c),\ I(s,c,x),\ G_e(u,s,c,x) \vdash varexp(s,c,x) \in \mathbb{N}$$

PO produced (set-valued variant).

$$AX(s,c),\ I(s,c,x),\ G_e(u,s,c,x) \vdash \mathrm{finite}(varexp(s,c,x))$$

R10 e/VAR: decrease of the variant

Trigger. A *convergent* event e.

PO produced (numeric variant).

$$AX(s,c),\ I,\ G_e,\ BAP(u,s,c,x,x') \vdash varexp(s,c,x') < varexp(s,c,x)$$

PO produced (set-valued variant).

$$AX(s,c),\ I,\ G_e,\ BAP(u,s,c,x,x') \vdash varexp(s,c,x') \subset varexp(s,c,x)$$

For an anticipated event. The variant is only required not to increase: $\leq$ (resp. $\subseteq$). See R18.

6 Rules for refinement

6.1 Reference definition and derivation

Definition (refinement of two events, *wp* style)

c refines a with respect to x, I, y, J if

$$AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \Rightarrow [\mathsf{c}]\,\neg[\mathsf{a}]\,\neg J(s,c,x,y).$$

This is the definition of operation refinement in B, transposed to events. The chapter transforms it in seven steps, all of them equivalences: unfolding [a] (normalised form with $\exists u$), simplifying the negation, unfolding [c] (with $\exists v$), eliminating $\forall y'$ (free variable), implication introduction (S1), eliminating $\exists v$ in the hypothesis (S3). With $\Gamma_0 \widehat{=} AX(s,c), I(s,c,x), J(s,c,x,y)$, one obtains:

$$(7) \qquad \Gamma_0,\ H(v,s,c,y),\ CBAP(v,s,c,y,y') \vdash \exists x' \cdot \exists u \cdot (G(u,s,c,x) \wedge ABAP(u,s,c,x,x') \wedge J(s,c,x',y')).$$

The definition does not guarantee the feasibility of c: it is proved separately (c/act$_i$/FIS, R16).

Property (event refinement, version II): GRD and SIM

c refines a *if, and only if,* (7) holds. Consequently, if c refines a, then

$$\begin{aligned} &\text{(GRD)} && AX(s,c), I, J, H(v,s,c,y) \vdash \exists u \cdot G(u,s,c,x) \\ &\text{(SIM)} && AX(s,c), I, J, H(v,s,c,y), CBAP(v,s,c,y,y') \\ & && \quad \vdash \exists x' \cdot \exists u \cdot ABAP(u,s,c,x,x') \wedge J(s,c,x',y') \end{aligned}$$

The converse does not hold in general: GRD and SIM are *necessary* but not *sufficient*.

Going from (7) to GRD and SIM separates two existentials that referred to the *same* value of u. Counterexample: $u \in \{0,1\}$, $G \equiv u = 0$, $ABAP \equiv u = 1 \wedge x' = x + 1$, $J \equiv true$; GRD holds with $u = 0$, SIM with $u = 1$, but (7) is false. In GRD, the hypothesis $CBAP$ is dropped: the abstract guard does not depend on y'; this is the form used by Rodin. The chapter's summary list (property `pro:poraf`, (l. 2244)) now follows this form for GRD, GRD-WIT and WFIS-P.

The witnesses restore the link. Rule S4 is applied with the witnesses declared in the witness clause of the concrete event: $u : WP(u,s,c,v,y)$ and $x' : WV(v,s,c,y',x')$.

Property (refinement with witnesses, version III)

If the POs GRD-WIT (R11), SIM-WIT (R12 and R13), WFIS-P and WFIS-V (R14) are valid, then c refines a.

Proof (chapter). WFIS-P gives u_0 with $WP(u_0)$, WFIS-V gives x'_0 with $WV(x'_0)$; GRD-WIT gives $G(u_0)$ and SIM-WIT gives $ABAP(u_0, x'_0) \wedge J(x'_0, y')$, hence (7). The key point is that GRD-WIT and SIM-WIT share the hypothesis $WP(u, s, c, v, y)$: the same value of u is used for both. The witnesses therefore make the decomposition sound; they are not merely an aid for the prover.

When is a witness required? For every abstract parameter that disappears in the concrete event, and for every abstract variable that disappears and that the abstract event modifies non-deterministically. Elsewhere, witnesses are optional but help the prover (see `temoins-obligatoires.pdf`).

Machine refinement. CM refines AM if the concrete initialisation refines the abstract one (R15) and if every event of CM refines an event of AM, the new events refining skip (l. 2397).

6.2 Rules generated for a concrete event

R11 c/grd/GRD: guard strengthening

Trigger. A concrete event c refines a, and each abstract guard $grd : G$.

PO produced (with a witness).

$$AX(s,c),\ I(s,c,x),\ J(s,c,x,y),\ H(v,s,c,y),\ WP(u,s,c,v,y) \vdash G(u,s,c,x)$$

Role. When the concrete event is observable, so is the abstract event: the concrete model shows no new behaviour.

R12 c/jnv$_i$/INV: preservation of the gluing invariant

Trigger. c and each invariant $jnv_i : J_i$ of the concrete machine.

Derivation. The conclusion of SIM-WIT, $ABAP \wedge J'$, is split by S2 into an INV part and a SIM part.

PO produced (l. 2665).

$$AX(s,c),\ I,\ J,\ H(v,s,c,y),\ CBAP(v,s,c,y,y'),\ WP(u,s,c,v,y),\ WV(v,s,c,y',x') \vdash J_i(s,c,x',y')$$

R13 c/act/SIM: simulation of the abstract action

Trigger. c and each abstract action $act : x :| ABAP$ of a.

PO produced.

$$AX(s,c),\ I,\ J,\ H(v,s,c,y),\ CBAP(v,s,c,y,y'),\ WP(u,s,c,v,y),\ WV(v,s,c,y',x') \vdash ABAP(u,s,c,x,x')$$

Role. Every concrete transition corresponds to an abstract transition. The abstract invariant I is not *proved again*: it is inherited, since a already preserves it.

R14 c/u/WFIS, c/x'/WFIS, c/u/WWD and c/x'/WWD: feasibility and well-definedness of witnesses

Trigger. Each witness declared in the witness clause.

Feasibility POs (non-deterministic witnesses).

$$AX(s,c),\ I,\ J,\ H(v,s,c,y) \vdash \exists u \cdot WP(u,s,c,v,y)$$
$$AX(s,c),\ I,\ J,\ H(v,s,c,y),\ CBAP(v,s,c,y,y') \vdash \exists x' \cdot WV(v,s,c,y',x')$$

Well-definedness POs (all witnesses) (l. 2705–2716).

$$AX(s,c),\ I,\ J,\ H(v,s,c,y) \vdash WD(WP(u,s,c,v,y))$$
$$AX(s,c),\ I,\ J,\ H(v,s,c,y),\ CBAP(v,s,c,y,y') \vdash WD(WV(v,s,c,y',x'))$$

Role. The WFIS POs make the use of witnesses sound: together with R11 to R13, they imply (7) (rule S4). The WWD POs treat a witness like any other formal text written by the modeller.

In Rodin: limitations and adjustments

Box *witnesses in Rodin* of the chapter (l. 2723–2736):

- Rodin checks statically that a witness is given for every abstract parameter that disappears and for every disappearing abstract variable that the abstract event modifies non-deterministically; a missing witness is reported as an error, before any proof;
- WFIS is generated only for non-deterministic witnesses: for a deterministic witness $u = E$ or $x' = E$, Rodin substitutes E for the variable in the other POs (one-point rule); only WWD remains;
- WWD is generated for all witnesses;
- limitation: the tool does not help in finding a witness; a wrong choice shows up only as an unprovable GRD, SIM, INV or WFIS PO, and the modeller must then revise the witness or the gluing invariant.

R15 Refined initialisation: INITIALISATION/jnv$_i$/INV, act/SIM, act$_i$/FIS, act$_i$/WD, x'/WFIS, x'/WWD

Global condition. $AX(s,c), CInit(s,c,y') \vdash \exists x' \cdot AInit(s,c,x') \land J(s,c,x',y')$.

With the witness $x' : WV(s,c,y',x')$ and S2. The initialisation has no parameter: the witness does not depend on v.

$AX(s,c),\ CInit(s,c,y'),\ WV(s,c,y',x') \vdash J_i(s,c,x',y')$	(INITIALISATION/jnv$_i$/INV)
$AX(s,c),\ CInit(s,c,y'),\ WV(s,c,y',x') \vdash AInit(s,c,x')$	(INITIALISATION/act/SIM)
$AX(s,c) \vdash \exists y' \cdot CInit(s,c,y')$	(INITIALISATION/act/FIS)
$AX(s,c) \vdash WD(CInit_i(s,c,y_i'))$	(INITIALISATION/act$_i$/WD)
$AX(s,c),\ CInit(s,c,y') \vdash \exists x' \cdot WV(s,c,y',x')$	(INITIALISATION/x'/WFIS)
$AX(s,c),\ CInit(s,c,y') \vdash WD(WV(s,c,y',x'))$	(INITIALISATION/x'/WWD)

x'/WFIS is produced only when the witness is not deterministic (l. 2612–2616).

R16 c/act$_i$/FIS, c/grd$_i$/WD, c/act$_i$/WD: feasibility and well-definedness of the concrete event

Trigger. The guards and actions of the concrete event c.

POs produced (l. 2744–2752). They are those of an abstract machine, with the abstract and gluing invariants as hypotheses:

$$AX(s,c),\ I,\ J,\ H(v,s,c,y) \vdash \exists y_i' \cdot CBAP_i(v,s,c,y,y_i') \qquad (\mathsf{c/act}_i\mathsf{/FIS})$$
$$AX(s,c),\ I,\ J,\ H_1,\ldots,H_{i-1} \vdash WD(H_i(v,s,c,y)) \qquad (\mathsf{c/grd}_i\mathsf{/WD})$$
$$AX(s,c),\ I,\ J,\ H(v,s,c,y) \vdash WD(CBAP_i(v,s,c,y,y_i')) \qquad (\mathsf{c/act}_i\mathsf{/WD})$$

As in R7, c/act$_i$/FIS is produced only for non-deterministic actions.

R17 th/THM: theorem of the concrete machine

$$AX(s,c),\ Th(s,c),\ I(s,c,x),\ J(s,c,x,y) \vdash SAFE(s,c,x,y)$$

R18 c/NAT, c/FIN, c/VAR: variants in a refinement

Trigger. A concrete event with status convergent or anticipated; the variant refers only to the concrete variables y.

$$AX(s,c), I, J, H(v,s,c,y) \vdash varexp(s,c,y) \in \mathbb{N} \qquad (\mathsf{c/NAT})$$
$$AX(s,c), I, J, H(v,s,c,y) \vdash \text{finite}(varexp(s,c,y)) \qquad (\mathsf{c/FIN},\ \text{set-valued variant})$$
$$AX(s,c), I, J, H, CBAP \vdash varexp(s,c,y') < varexp(s,c,y) \qquad (\text{convergent})$$
$$AX(s,c), I, J, H, CBAP \vdash varexp(s,c,y') \leq varexp(s,c,y) \qquad (\text{anticipated})$$

Rodin produces one or the other c/VAR PO depending on the status of the event; for a set-valued variant, $<$ and $\leq$ become $\subset$ and $\subseteq$ (l. 2799).

Methodology. A *new* event implicitly refines skip: its SIM PO then reduces to $x' = x$. It must be convergent so as not to take control forever. An anticipated event postpones this obligation to a later refinement, while guaranteeing that it does not increase the variant.

R19 c/MRG: event merging

Trigger. A concrete event c that refines *two* abstract events a$_1$ and a$_2$ (c(a1+a2)).

Application condition (syntactic). The actions of a$_1$, a$_2$ and c must be *syntactically identical*, and the merged events must have the same parameters.

PO produced.

$$AX(s,c),\ I(s,c,x),\ J(s,c,x,y),\ H(v,s,c,y) \vdash G_1(u,s,c,x) \vee G_2(u,s,c,x)$$

The gluing invariant J links y (in H) to x (in G_1, G_2).

Role. The GRD PO is replaced by the strengthening of the *disjunction* of the guards. The rule makes it possible to reduce the number of events, for instance to obtain a conditional. In the example of the maximum of two numbers (machines MRG0 to MRG3), the PO becomes $done = FALSE \vdash (i < j \wedge done = FALSE) \vee (i \geq j \wedge done = FALSE)$, which reduces to $i < j \vee i \geq j$, hence to true.

7 Deadlock freedom (semantics)

These POs are *not* generated systematically by Rodin or by Atelier B: the modeller adds them as theorems, or checks them by model checking (ProB).

R20 m/DLF: deadlock-free machine

$$AX(s,c),\ I(s,c,x) \vdash \bigvee_{\mathsf{e}\in E(\mathsf{AM})} \exists u \cdot G_{\mathsf{e}}(u,s,c,x)$$

R21 r/DLF: the refinement does not deadlock more often

$$AX(s,c),\ I,\ J,\ \bigvee_{\mathsf{e}\in E(\mathsf{AM})} \exists u \cdot G_{\mathsf{e}}(u,s,c,x) \vdash \bigvee_{\mathsf{e}\in E(\mathsf{CM})} \exists v \cdot H_{\mathsf{e}}(v,s,c,y)$$

8 Summary table

Name	Trigger	Hypotheses $\vdash$ goal
x/WD	any expression	$AX(s,c)\,[,I,G] \vdash WD(x)$
th/THM (context)	context theorem	$AX(s,c), Th \vdash th$
th_i/THM (machine)	machine theorem	$AX(s,c), Th_i, I \vdash SAFE_i$
INITIALISATION/inv_i/INV	invariant × init.	$AX(s,c), initBAP' \vdash I'_i$
INITIALISATION/act_i/FIS	non-det. init. action	$AX(s,c) \vdash \exists x'_i \cdot initBAP_i$
INITIALISATION/act_i/WD	init. action	$AX(s,c) \vdash WD(initBAP_i)$
e/inv_i/INV	invariant × event	$AX(s,c), I, G, BAP \vdash I'_i$
e/act_i/FIS	non-det. action	$AX(s,c), I, G \vdash \exists x'_i \cdot BAP_i$
e/grd_i/WD	guard	$AX(s,c), I, G_1..G_{i-1} \vdash WD(G_i)$
e/act_i/WD	action	$AX(s,c), I, G \vdash WD(BAP_i)$
e/NAT	convergent/anticipated event	$AX(s,c), I, G \vdash varexp \in \mathbb{N}$
e/FIN	same, set-valued variant	$AX(s,c), I, G \vdash \text{finite}(varexp)$
e/VAR	convergent event	$AX(s,c), I, G, BAP \vdash varexp' < varexp$ (or $\subset$)
c/grd/GRD	abstract guard × refinement	$AX(s,c), I, J, H, WP \vdash G$
c/act/SIM	abstract action × refinement	$AX(s,c), I, J, H, CBAP, WP, WV \vdash ABAP$
c/jnv_i/INV	gluing invariant	$AX(s,c), I, J, H, CBAP, WP, WV \vdash J'_i$
c/u/WFIS	non-det. parameter witness	$AX(s,c), I, J, H \vdash \exists u \cdot WP$
c/x'/WFIS	non-det. variable witness	$AX(s,c), I, J, H, CBAP \vdash \exists x' \cdot WV$
c/u/WWD, c/x'/WWD	any witness	same hypotheses $\vdash WD(WP)$, $WD(WV)$
c/act_i/FIS	non-det. concrete action	$AX(s,c), I, J, H \vdash \exists y'_i \cdot CBAP_i$
c/grd_i/WD, c/act_i/WD	concrete guard, action	$AX(s,c), I, J, H_1..H_{i-1} \vdash WD(H_i)$; $AX(s,c), I, J, H \vdash WD(CBAP_i)$
INITIALISATION/jnv_i/INV	refined init.	$AX(s,c), CInit', WV \vdash J_i(x', y')$
INITIALISATION/act/SIM	refined init.	$AX(s,c), CInit', WV \vdash AInit(x')$

Name	**Trigger**	**Hypotheses $\vdash$ goal**
INITIALISATION/x'/WFIS, x'/WWD	init. witness	$AX(s,c), CInit' \vdash \exists x' \cdot WV$; $\vdash WD(WV)$
th/THM (refinement)	concrete theorem	$AX(s,c), Th, I, J \vdash SAFE$
c/NAT, c/FIN	convergent / anticipated	$AX(s,c), I, J, H \vdash varexp(y) \in \mathbb{N}$ / finite
c/VAR	convergent / anticipated	$\cdots \vdash varexp(y') < varexp(y)$ / $\leq$
c/MRG	merging $\mathsf{a}_1 + \mathsf{a}_2$	$AX(s,c), I, J, H \vdash G_1 \vee G_2$
m/DLF, r/DLF	optional	disjunction of the guards

References

[1] Dominique Méry. *Modelling Software-based Systems: Correct by Construction Paradigm*, chapter The Event-B modelling language. ISTE, 2026.

[2] Dominique Méry and Neeraj Kumar Singh. *Modelling Software-based Systems: Correct by Construction Paradigm*. ISTE, 2026 to appear.